

Indexing All (r, s) -Nucleus Decompositions

Anonymous Author(s)

Abstract

Dense subgraphs carry much of the meaning of a graph, and decomposing a graph into hierarchies of dense subgraphs is a fundamental task in graph mining. The (r, s) -nucleus decomposition is the clique-level member of this family, scoring every r -clique by its surrounding s -cliques. It subsumes both the k -core and the k -truss. The parameter pair (r, s) acts as a resolution, and analysts need many cells of the parameter plane rather than one. Existing algorithms compute one cell at a time and materialize every r -clique of that cell. A plane is therefore out of reach: a seven-million-edge web graph already contains more than 10^{12} 5-cliques. We observe that vertices belonging to exactly the same maximal cliques are interchangeable, and that the whole plane respects this symmetry. Based on this observation, we propose NSI, an index built on the clique quotient of the graph. Cliques collapse into patterns, support counts collapse into polynomial coefficients, and two transfer theorems reduce the whole plane to a single peeled cell. Wherever the closed form is certified, the index stores nothing and reconstructs the answer at query time. On real graphs the index is up to seven orders of magnitude smaller than the explicit table it replaces, and it answers point queries in tens of nanoseconds. Its builder outperforms the state-of-the-art single-cell algorithm by up to $742\times$ in time and $226\times$ in memory. A one-number test, computable before any index is built, tells in advance whether a graph is suitable.

1 Introduction

A graph represents entities as vertices and their pairwise relationships as edges. Groups of vertices that are densely connected to one another often carry the meaning of the whole network. For example, in collaboration networks, dense groups correspond to research teams working closely together [5]; in web graphs, they mark clusters of pages on one topic [4]; in biological networks, they capture protein complexes [10]; in financial and social platforms, they expose coordinated fraud rings [1, 6].

Decomposing a graph into a hierarchy of nested dense subgraphs is a fundamental task in graph mining. Representative applications include community detection [5, 11], densest-subgraph discovery [3], influence analysis [7, 8], and graph visualization [9]. In the literature, the task has been studied at three levels of granularity. The k -core peels vertices by degree [2, 15]; the k -truss peels edges by triangle count [16, 18]; the (r, s) -nucleus decomposition peels r -cliques by s -clique count [13]. The nucleus model subsumes the other two: the $(1, 2)$ -nucleus is exactly the k -core, and the $(2, 3)$ -nucleus is exactly the k -truss [13]. The parameter pair (r, s) therefore acts as a resolution, and every cell of the parameter plane reveals the graph’s dense structure at a different granularity [13, 14]. The state-of-the-art algorithm, CND [17], computes the decomposition of one cell at a time. In this paper, we aim to answer every cell of a bounded parameter plane from one index.

For integers $r < s$, consider subgraphs in which every r -clique is contained in at least k s -cliques of the same subgraph. The nucleus value of an r -clique is the largest k for which such a subgraph

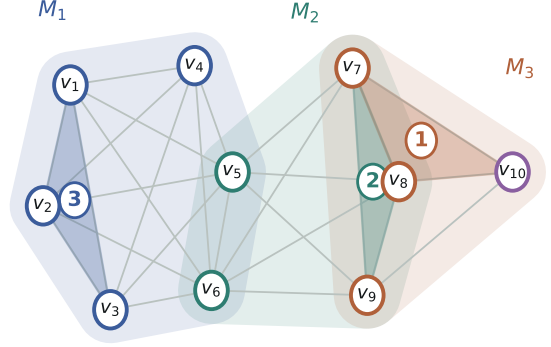


Figure 1: An example graph G and three of its $(3, 4)$ -nucleus values.

contains it. Computing the cell (r, s) means computing this value for every r -clique of the graph.

Example 1.1. Consider the graph G in Figure 1, whose ten vertices form three maximal cliques: M_1 with six vertices, M_2 with five, and M_3 with four. At the cell $(3, 4)$, the triangle $\{v_1, v_2, v_3\}$ has nucleus value 3: it lies in the subgraph induced by M_1 , in which every triangle is contained in at least three 4-cliques. The triangle $\{v_7, v_8, v_9\}$ has value 2, and the triangle $\{v_7, v_8, v_{10}\}$ has value 1. Other cells give other resolutions: at $(3, 5)$ the value of $\{v_1, v_2, v_3\}$ is still 3, while at $(4, 5)$ the 4-clique $\{v_1, v_2, v_3, v_4\}$ has value 2.

Applications. Many uses of the decomposition need several cells of the plane rather than one. *Multi-resolution exploration.* An analyst inspecting a dense region rarely knows the right resolution in advance. Low cells surface broad communities, while high cells isolate small, strongly coordinated groups [6, 13]. Exploration therefore issues a stream of queries across cells, and each query must return in interactive time. *Parameter selection.* Which cell best separates signal from noise differs across graphs and tasks [14]. Practitioners choose by probing: compute several cells, compare the resulting hierarchies, and keep the most informative one. Today every probe costs a full decomposition run. *Point queries at query time.* Downstream systems ask point questions: given one concrete group of vertices, how dense is its surrounding structure at each resolution [12]? Answering from the raw graph requires a fresh decomposition per question.

Existing Methods and Key Limitation. The state-of-the-art algorithm CND [17] enumerates every r -clique of the requested cell, counts its supporting s -cliques, and repeatedly removes the clique of minimum count. Careful counting structures let it obtain these counts without listing individual s -cliques, which made single-cell decomposition practical on graphs with millions of edges [17]. However, all existing algorithms share a fundamental limitation: they materialize *every r -clique of every cell they compute*. The number of r -cliques grows combinatorially with r : web-it-2004, a web graph

with 7.2 million edges, contains more than 1.5×10^{12} 5-cliques. Even a single shallow cell is already prohibitive: on web-it-2004, CND needs 5,581 seconds and 101 GB of memory for the single cell (3, 4). Within a 1,800-second, 300 GB budget, CND finishes none of the nine web-it-2004 cells of our evaluation grid (Section 8).

This leads to the central research question of this paper: **Can one index answer the nucleus value of every r -clique, at every cell of a bounded plane, without ever materializing the plane?** **Challenges.** Answering the plane from one structure faces three obstacles. Astronomically many objects. For one small cell, per-clique computation is affordable. However, across a plane the objects multiply twice: the number of r -cliques explodes with r , and every cell owns its own copy of the work. Even the ten-vertex graph of Figure 1 carries 61 cliques of size three to five; on web-it-2004 the count exceeds 10^{12} . Therefore, any approach that touches each r -clique of each cell even once is ruled out from the start. Counting without enumeration.

Peeling is driven by counts: each removal must decrement the count of every r -clique that shared a supporting s -clique with the removed one. However, s -cliques outnumber r -cliques, and at high s they cannot even be listed once. Therefore, both the initial counts and every later decrement must be obtained without enumerating a single s -clique. A plane of cells. A bounded plane contains dozens of cells, and each cell is a full decomposition in its own right. However, running even the best single-cell algorithm once per cell multiplies its cost by the number of cells. Worse, the deep cells are exactly the ones no existing algorithm reaches. Therefore, the computation done in one cell must be made to pay for all the others.

Our Idea. Our solution is built on one structural observation: vertices that belong to exactly the same maximal cliques are interchangeable, and the whole plane respects this symmetry. We call the resulting grouping the *clique quotient* of the graph. Peel patterns, not cliques.

Under the quotient, an r -clique is described by how many vertices it takes from each group, not by which vertices it takes. The 33 triangles of Figure 1 collapse into 7 such patterns; on real graphs the collapse reaches six orders of magnitude. The peel runs unchanged on patterns, with every step carrying a multiplicity. Count as a coefficient. The number of s -cliques containing a pattern is a coefficient of one small polynomial derived from the quotient. Both the initial counts and all later decrements reduce to coefficient extraction, so no s -clique is ever listed. Certify instead of recompute. Two transfer theorems link neighboring cells. Along s , a pattern whose value reaches its closed form keeps that form in every later cell. Along r , the starting values of a cell follow from the cell below it. Together they leave a single cell to peel; every other cell of the plane is arithmetic.

The consequence is an index that stores almost nothing. Whenever the closed form is certified, the value is reconstructed at query time from the quotient itself, so certified patterns cost zero bytes. We call the structure NSI, for Nucleus Spectrum Index.

Contributions. Our key contributions are summarized as follows. The first index over the nucleus plane. We propose NSI (Section 7), the first index that answers every cell of a bounded (r, s) plane from one structure. On web-it-2004, the explicit table behind the same queries would occupy 46.2 TB, and it cannot be built at all for $r \geq 4$. NSI occupies 1.66 MB and answers a point query in 74

Table 1: Notation used across sections.

Symbol	Meaning
$G = (V, E)$	input graph, $n = V $, $m = E $
r, s	scored-clique size and witness-clique size, $r < s$
R, S	an r -clique (scored) and an s -clique (witness)
$\deg_{(s)}(R)$	s -clique support of R (Definition 2.2)
$\kappa_s(R)$	core number of R at witness size s (Definition 2.4)
$\mathcal{M}(G)$	the maximal cliques of G
$\omega(R)$	local clique number: size of the largest clique containing R
$\mathcal{P}$	the indexed plane: $r \in [r_{\min}, r_{\max}]$, $r < s \leq s_{\max}$

nanoseconds. A quotient framework with certified transfer. We develop the clique quotient and its peel (Sections 4–6): interchangeability, coefficient counting, and two transfer certificates that reduce the whole plane to one peeled cell. Disabling the certificates alone slows the build by 22.6 $\times$ on nasasrb, with answers unchanged. An applicability test computable in advance. We characterize exactly when the quotient compresses, by a one-number test computable before any index is built (Section 7). The test predicted every graph of our evaluation, and it also predicted the social networks the evaluation excludes. On soc-pokec the quotient compresses by a factor of only 1.001, and the test reports this without building anything. Comprehensive Experimental Evaluation. We evaluate on eight graphs from four domains, all with at least a million edges (Section 8). Against CND run serially on the same machine, our builder wins every cell of the evaluation grid, by up to 742 $\times$ in time and 226 $\times$ in memory. It also completes the deep cells on which CND exceeds every budget.

2 Preliminaries

In this section, we fix notation, formalize the core number of an r -clique, and state the plane-indexing problem.

Graph notation. We consider a simple, undirected, unweighted graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges. For $V' \subseteq V$, we write $G[V']$ for the subgraph of G induced by V' . Table 1 summarizes the notation used across sections; symbols local to one lemma or algorithm are introduced where they are used.

Definition 2.1 (ℓ -Clique). For an integer $\ell > 0$, a vertex set $X \subseteq V$ with $|X| = \ell$ is an ℓ -clique of G if every two distinct vertices of X are adjacent.

Following the convention of nucleus decomposition [13, 17], we call an r -clique a *scored clique*: the decomposition computes a core number for it. An s -clique is a *witness clique*: it certifies the cohesion of the scored cliques it contains.

Maximal cliques. A clique is *maximal* if no outside vertex can be added to it while preserving a clique; $\mathcal{M}(G)$ denotes the set of maximal cliques of G . Every clique of G is contained in at least one maximal clique. We write $\omega(R)$ for the size of the largest clique of G that contains the clique R , and call it the *local clique number* of R . In Figure 1, $\omega(\{v_1, v_2, v_3\}) = 6$ by M_1 , $\omega(\{v_7, v_8, v_9\}) = 5$ by M_2 , and $\omega(\{v_7, v_8, v_{10}\}) = 4$ by M_3 .

Definition 2.2 (s -Clique Support). The *s -clique support* of an r -clique R in G is $\deg_{(s)}(R) = |\{S \subseteq V : R \subseteq S, S \text{ is an } s\text{-clique of } G\}|$.

In Figure 1, $\deg_{(4)}(\{v_1, v_2, v_3\}) = 3$: the three 4-cliques obtained by adding v_4, v_5 , or v_6 .

Nucleus and core number. Cohesion is certified by a family of witness cliques that sustains all of its own scored cliques [13]. For a family $\mathcal{S}$ of s -cliques, write $R(\mathcal{S})$ for the set of r -cliques contained in at least one member of $\mathcal{S}$. Write $G[\mathcal{S}]$ for the subgraph induced by the union of the members.

Definition 2.3 (k -(r, s)-Nucleus). A k -(r, s)-nucleus of G is a subgraph $G[\mathcal{S}]$ induced by a family $\mathcal{S}$ of s -cliques such that (i) every $R \in R(\mathcal{S})$ is contained in at least k members of $\mathcal{S}$; (ii) any two members are linked inside $\mathcal{S}$ by a chain of members in which consecutive cliques share at least r vertices; and (iii) $\mathcal{S}$ is maximal under inclusion among families satisfying (i) and (ii).

Support in condition (i) is counted within the family, not in G : a nucleus must sustain its own scored cliques. Condition (ii) links consecutive witnesses through shared scored cliques, since two s -cliques share r vertices exactly when they contain a common r -clique.

Definition 2.4 (Core Number). The *core number* $\kappa_s(R)$ of an r -clique R is the largest k such that $R \in R(\mathcal{S})$ for the defining family $\mathcal{S}$ of some k -(r, s)-nucleus, and 0 if R lies in no s -clique.

The size of the scored clique is implicit in its argument, so the single notation $\kappa_s(R)$ serves every cell of the plane. The nucleus model subsumes the classical models: $(r, s) = (1, 2)$ yields the k -core [15] and $(r, s) = (2, 3)$ yields the k -truss [13, 18]. In Figure 1, $\kappa_4(\{v_1, v_2, v_3\}) = 3$, as computed in Example 1.1.

Problem statement (plane indexing). A *plane* $\mathcal{P}$ is the set of cells (r, s) with $r_{\min} \leq r \leq r_{\max}$ and $r < s \leq s_{\max}$. Given G and $\mathcal{P}$, the problem is to build an index that, for any s and any r -clique R with $(|R|, s) \in \mathcal{P}$, returns $\kappa_s(R)$. A scored clique contained in no witness has core number 0. The algorithmic sections therefore focus on scored cliques that lie in at least one maximal clique of size at least s .

The next section recalls how the state of the art computes one cell, and measures where its cost concentrates.

3 The State-of-the-Art Approach

In this section, we recall the peeling skeleton that all existing algorithms instantiate, and we measure where its cost concentrates.

The peeling skeleton. Every existing algorithm computes one cell (r, s) by the same skeleton, shown in Algorithm 1 [13, 17]. It materializes the scored cliques, initializes their supports, and repeatedly removes a clique of minimum support. The level k never decreases, and each clique receives the level at which it is removed.

Algorithm 1 enumerates the scored cliques and allocates one mutable record per clique (Line 1). It initializes each support (Line 3) and then peels: extract a minimum (Line 6), assign the current level (Line 7), and credit every scored clique that loses a witness (Lines 8–10). The state of the art, CND [17], realizes Lines 3 and 8–10 by aggregated counting, so no witness clique is ever listed one by one. This counting is what made single cells with millions of edges practical, and we adopt it as our baseline throughout.

Cost anatomy. The skeleton’s remaining cost sits in Line 1: every scored clique of the cell exists as a record, and every record carries mutable peel state. Memory is therefore at least proportional to

Algorithm 1: The explicit peel of existing algorithms.

```

Input: Graph  $G$ , cell  $(r, s)$ 
Output:  $\kappa_s(R)$  for every  $r$ -clique  $R$  of  $G$ 
1  $\mathcal{R} \leftarrow$  all  $r$ -cliques of  $G$ ; // one record per clique
2 foreach  $R \in \mathcal{R}$  do
3    $\text{sup}(R) \leftarrow \deg_{(s)}(R)$ 
4  $k \leftarrow 0$ ;
5 while  $\mathcal{R} \neq \emptyset$  do
6    $R^* \leftarrow$  a clique of minimum  $\text{sup}$  in  $\mathcal{R}$ ;
7    $k \leftarrow \max(k, \text{sup}(R^*))$ ;  $\kappa_s(R^*) \leftarrow k$ ;
8   foreach witness  $S \supseteq R^*$  not yet destroyed do
9     foreach  $R' \subset S$  with  $R' \in \mathcal{R}$ ,  $R' \neq R^*$  do
10       $\text{sup}(R') \leftarrow \text{sup}(R') - 1$ 
11   remove  $R^*$  from  $\mathcal{R}$ ;
12 return  $\kappa_s$ 

```

Table 2: Scored cliques on web-it-2004 (7.2 million edges).

r	# r -cliques	one record each
3	338,786,461	2.7 GB
4	28,530,662,583	228 GB
5	2,140,698,950,272	17 TB

the number of scored cliques, and time to the stream of per-clique updates. Both grow combinatorially with r , as Table 2 shows for web-it-2004.

The second column of Table 2 counts the objects of Line 1; the third charges a conservative eight bytes per record. At $r = 4$ the records alone exceed the memory of a large server, and at $r = 5$ they exceed it by two orders of magnitude. The measured behavior matches: on web-it-2004, CND spends 5,581 seconds and 101 GB on the single shallow cell $(3, 4)$. Within a 1,800-second, 300 GB budget, it finishes none of the nine cells of our evaluation grid (Section 8).

What a plane demands. The skeleton is also strictly per cell: nothing computed at (r, s) is reused at $(r, s+1)$ or $(r+1, s)$. A plane therefore multiplies the cost by its number of cells. Aggregated counting removed the witness side of the cost; the scored side, one record per clique per cell, remains. Removing it requires a representation in which many scored cliques share one record and all cells share one representation. Building that representation is the subject of the next section.

4 The Clique Quotient

Algorithm 1 pays once per scored clique, so its cost is fixed by how many scored cliques exist. This section asks when two scored cliques are genuinely different, and shows that on real graphs most of them are not. We first group vertices, then group cliques, and finally quantify the collapse.

Classes. For a vertex v , let $\mathcal{M}(v) = \{M \in \mathcal{M}(G) : v \in M\}$ denote the maximal cliques that contain it.

Definition 4.1 (Class). A *class* of G is a maximal set of vertices that all have the same maximal-clique membership: u and v belong to one class if and only if $\mathcal{M}(u) = \mathcal{M}(v)$.

Classes partition V , and the number of classes never exceeds n . In Figure 1, the classes are $A = \{v_1, \dots, v_4\}$ inside M_1 only; $B = \{v_5, v_6\}$ shared by M_1 and M_2 ; $C = \{v_7, v_8, v_9\}$ shared by M_2 and M_3 ; and $D = \{v_{10}\}$ inside M_3 only. The vertex colors of the figure encode

exactly this partition. Classes sit cleanly inside maximal cliques, which the following lemma records.

LEMMA 4.2 (CLASS CONTAINMENT). *Let c be a class and M a maximal clique. Then either every vertex of c belongs to M , or none does.*

PROOF. Suppose $u \in c$ lies in M , and let $v \in c$ be any other vertex of the class. Then $M \in \mathcal{M}(u) = \mathcal{M}(v)$, so $v \in M$. $\square$

Interchangeability. Vertices of one class are not merely similar; they are indistinguishable by the graph itself.

THEOREM 4.3 (INTERCHANGEABILITY). *Let $u \neq v$ be vertices of one class. Then the transposition that swaps u and v and fixes every other vertex is an automorphism of G . Consequently, for every s and every r -clique R with $u \in R$ and $v \notin R$, $\kappa_s(R) = \kappa_s(R \setminus \{u\} \cup \{v\})$.*

PROOF. Let M be any maximal clique containing u ; since $\mathcal{M}(u) = \mathcal{M}(v)$, it also contains v , so u and v are adjacent. Let w be any neighbor of u other than v . The edge $\{u, w\}$ lies in some maximal clique M' , and $M' \in \mathcal{M}(u) = \mathcal{M}(v)$ places v in M' , so w is also a neighbor of v . Hence $N[u] \subseteq N[v]$, and by symmetry $N[u] = N[v]$. A transposition of two adjacent vertices with equal closed neighborhoods preserves adjacency, so it is an automorphism. Core numbers depend only on the isomorphism type of G , so every automorphism preserves κ_s . Applying the transposition to R yields $R \setminus \{u\} \cup \{v\}$, which proves the claim. $\square$

In Figure 1, swapping v_1 and v_2 maps the graph onto itself, so every core number that mentions v_1 has a twin that mentions v_2 . **Patterns.** Interchangeability licenses forgetting which vertices a scored clique uses.

Definition 4.4 (Pattern). The *pattern* of an r -clique R is the multiset of classes of its vertices, written $P = \{c_1^{b_1}, \dots, c_t^{b_t}\}$ with $b_1 + \dots + b_t = r$, where R takes b_i vertices from class c_i .

COROLLARY 4.5. *Two r -cliques with the same pattern have the same core number at every s .*

PROOF. Permuting vertices within one class is a product of same-class transpositions, each an automorphism by Theorem 4.3, so it is an automorphism. For two cliques with one pattern, some within-class permutation maps the first onto the second. Automorphisms preserve κ_s , which proves the claim. $\square$

Core numbers are therefore functions of patterns, and we write $\kappa_s(P)$ for the shared value. The peel of Algorithm 1 can run on patterns directly, provided each pattern knows how many cliques it stands for.

LEMMA 4.6 (PATTERN MULTIPLICITY). *Let $P = \{c_1^{b_1}, \dots, c_t^{b_t}\}$ be the pattern of at least one r -clique, and let $n_i = |c_i|$. Then the number of r -cliques with pattern P is exactly $\prod_{i=1}^t \binom{n_i}{b_i}$.*

PROOF. Some r -clique with pattern P lies in a maximal clique M , so each c_i intersects M , and Lemma 4.2 puts each c_i wholly inside M . All vertices of $c_1 \cup \dots \cup c_t$ are then pairwise adjacent, so every choice of b_i vertices from each c_i forms an r -clique, and each such clique has pattern P . The number of choices is $\prod_i \binom{n_i}{b_i}$. $\square$

Table 3: The patterns of G at $r = 3$.

pattern	multiplicity	#cliques
$\{A^3\}$	$\binom{4}{3}$	4
$\{A^2B\}$	$\binom{4}{2} \binom{2}{1}$	12
$\{AB^2\}$	$\binom{4}{1} \binom{2}{2}$	4
$\{B^2C\}$	$\binom{2}{2} \binom{3}{1}$	3
$\{BC^2\}$	$\binom{2}{1} \binom{3}{2}$	6
$\{C^3\}$	$\binom{3}{3}$	1
$\{C^2D\}$	$\binom{3}{2} \binom{1}{1}$	3
7 patterns		33

The collapse, quantified. Summing Lemma 4.6 over the realized patterns counts all scored cliques. The ratio of cliques to patterns is therefore the average multiplicity:

$$\frac{\#r\text{-cliques}}{\#\text{patterns}} = \text{avg}_P \prod_i \binom{n_i}{b_i}. \quad (1)$$

The ratio is driven by class sizes: one class of hundreds of vertices contributes binomials that dwarf any number of singleton classes.

Example 4.7. Consider the graph G in Figure 1 at $r = 3$. Its 33 triangles realize the 7 patterns of Table 3; for instance, all 12 triangles with pattern $\{A^2B\}$ share the core number computed once in Example 1.1. Across the whole plane $r \in [3, 5]$, $s \leq 8$, the 61 scored cliques of G realize 16 patterns.

On real graphs the collapse is far larger. On pkustk13, a finite-element mesh, 94,893 vertices form 31,872 classes, and the 2.1×10^{12} 5-cliques of web-it-2004 realize 1.3 million patterns, a collapse of six orders of magnitude. Section 7 returns to Equation (1) as a predictor: the average is computable before any decomposition, and it decides in advance whether the quotient is worth building.

Patterns are now the unit of peeling. The peel still needs their supports, and obtaining those without enumerating witnesses is the subject of the next section.

5 Peeling in the Quotient

6 From One Cell to the Plane

7 The Index and Its Query

8 Experiments

9 Related Work

10 Conclusion

References

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph-based anomaly detection and description: a survey. *Data Mining and Knowledge Discovery* 29, 3 (2015), 626–688.
- [2] Vladimir Batagelj and Matjaž Zaveršnik. 2011. Fast algorithms for determining (generalized) core groups in social networks. *Adv. Data Anal. Classif.* 5, 2 (July 2011), 129–145. doi:10.1007/s11634-010-0079-y
- [3] Pierre Dupont, Jérôme Callut, Grégoire Doooms, Jean-Noël Monette, and Yves Deville. 2006. Relevant subgraph extraction from random walks in a graph. <https://api.semanticscholar.org/CorpusID:15222702>
- [4] David Gibson, Jon Kleinberg, and Prabhakar Raghavan. 1998. Inferring Web communities from link topology. In *Proceedings of the Ninth ACM Conference on Hypertext and Hypermedia: Links, Objects, Time and Space—Structure in Hypermedia Systems: Links, Objects, Time and Space—Structure in Hypermedia Systems* (Pittsburgh, Pennsylvania, USA) (HYPERTEXT '98). Association for Computing Machinery, New York, NY, USA, 225–234. doi:10.1145/276627.276652

- [5] M. Girvan and M. E. J. Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826. arXiv:<https://www.pnas.org/doi/pdf/10.1073/pnas.122653799> doi:10.1073/pnas.122653799
- [6] Meng Jiang, Peng Cui, Alex Beutel, Christos Faloutsos, and Shiqiang Yang. 2016. Catching Synchronized Behaviors in Large Networks: A Graph Mining Approach. 10, 4 (2016). doi:10.1145/2746403
- [7] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, D.C.) (KDD '03). Association for Computing Machinery, New York, NY, USA, 137–146. doi:10.1145/956750.956769
- [8] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2015. Influential community search in large networks. *Proc. VLDB Endow.* 8, 5 (Jan. 2015), 509–520. doi:10.14778/2735479.2735484
- [9] Fragkiskos D. Malliaros and Michalis Vazirgiannis. 2013. Clustering and community detection in directed networks: A survey. *Physics Reports* 533, 4 (2013), 95–142. doi:10.1016/j.physrep.2013.08.002 Clustering and Community Detection in Directed Networks: A Survey.
- [10] Francis J O'Reilly, Andrea Graziadei, Christian Forbrig, Rica Bremenkamp, Kristine Charles, Swantje Lenz, Christoph Elfmann, Lutz Fischer, Jörg Stülke, and Juri Rappsilber. 2023. Protein complexes in cells by AI‐assisted structural proteomics. *Molecular Systems Biology* 19, 4 (2023), e11544. arXiv:<https://www.embopress.org/doi/pdf/10.15252/msb.202311544> doi:10.15252/msb.202311544
- [11] Giulio Rossetti and Rémy Cazabet. 2018. Community Discovery in Dynamic Networks: A Survey. *ACM Comput. Surv.* 51, 2, Article 35 (Feb. 2018), 37 pages. doi:10.1145/3172867
- [12] Ryan A. Rossi, Nesreen K. Ahmed, Rong Zhou, and Hoda Eldardiry. 2018. Interactive Visual Graph Mining and Learning, Vol. 9. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3200764
- [13] Ahmet Erdem Sariyuce, C. Seshadhri, Ali Pinar, and Umit V. Catalyurek. 2015. Finding the Hierarchy of Dense Subgraphs using Nucleus Decompositions. In *Proceedings of the 24th International Conference on World Wide Web* (Florence, Italy) (WWW '15). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 927–937. doi:10.1145/2736277.2741640
- [14] Ahmet Erdem Sariyuce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2017. Nucleus Decompositions for Identifying Hierarchy of Dense Subgraphs. *ACM Trans. Web* 11, 3, Article 16 (July 2017), 27 pages. doi:10.1145/3057742
- [15] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social Networks* 5, 3 (1983), 269–287. doi:10.1016/0378-8733(83)90028-X
- [16] Jia Wang and James Cheng. 2012. Truss decomposition in massive networks. *Proc. VLDB Endow.* 5, 9 (May 2012), 812–823. doi:10.14778/2311906.2311909
- [17] Wenqian Zhang, Zhengyi Yang, Dong Wen, Yi Ding, Wenjie Zhang, and Xuemin Lin. 2026. Nucleus Decomposition Revisited: An Efficient Counting-Based Approach. *Proc. ACM Manag. Data* 4, 3, Article 215 (May 2026), 26 pages. doi:10.1145/3802092
- [18] Feng Zhao and Anthony K. H. Tung. 2012. Large scale cohesive subgraphs discovery for social network visual analysis. *Proc. VLDB Endow.* 6, 2 (Dec. 2012), 85–96. doi:10.14778/2535568.2448942